Name: Phan Tran Thanh Huy
ID: ITCSIU22056

Lab 3

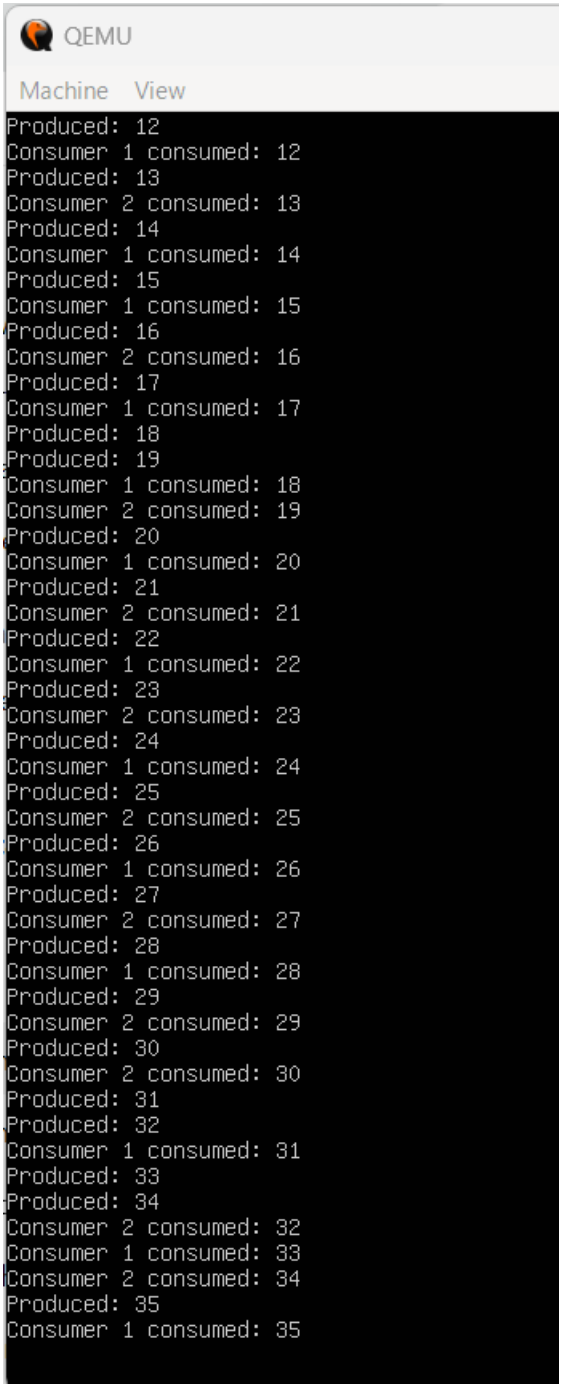
I51

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0, out = 0;

pthread_mutex_t mutex;
sem_t empty_slots;
sem_t full_slots;

int running = 1;

void* producer(void* arg) {
    int value = 1;
    while (running) {
        sem_wait(&empty_slots);
        pthread_mutex_lock(&mutex);

        buffer[in] = value;
        printf("Produced: %d\n", value);
        in = (in + 1) % BUFFER_SIZE;
        value++;

        pthread_mutex_unlock(&mutex);
        sem_post(&full_slots);

        usleep(rand() % 500000);
    }
    return NULL;
}
```

```
void* consumer(void* arg) {
    int id = *(int*)arg;
    while (running) {
        sem_wait(&full_slots);
        pthread_mutex_lock(&mutex);

        int value = buffer[out];
        printf("Consumer %d consumed: %d\n", id, value);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty_slots);

        usleep(rand() % 800000);
    }
    return NULL;
}
```

```
int main() {
    srand(time(NULL));

    pthread_t prod, cons[2];
    int cons_id[2] = {1, 2};

    pthread_mutex_init(&mutex, NULL);
    sem_init(&empty_slots, 0, BUFFER_SIZE);
    sem_init(&full_slots, 0, 0);

    pthread_create(&prod, NULL, producer, NULL);
    for (int i = 0; i < 2; i++)
        pthread_create(&cons[i], NULL, consumer, &cons_id[i]);

    sleep(10);
    running = 0;

    pthread_join(prod, NULL);
    for (int i = 0; i < 2; i++)
        pthread_join(cons[i], NULL);

    pthread_mutex_destroy(&mutex);
    sem_destroy(&empty_slots);
    sem_destroy(&full_slots);

    printf("Program finished.\n");
    return 0;
}
```

Result:


QEMU
Machine View
Produced: 12
Consumer 1 consumed: 12
Produced: 13
Consumer 2 consumed: 13
Produced: 14
Consumer 1 consumed: 14
Produced: 15
Consumer 1 consumed: 15
Produced: 16
Consumer 2 consumed: 16
Produced: 17
Consumer 1 consumed: 17
Produced: 18
Produced: 19
Consumer 1 consumed: 18
Consumer 2 consumed: 19
Produced: 20
Consumer 1 consumed: 20
Produced: 21
Consumer 2 consumed: 21
Produced: 22
Consumer 1 consumed: 22
Produced: 23
Consumer 2 consumed: 23
Produced: 24
Consumer 1 consumed: 24
Produced: 25
Consumer 2 consumed: 25
Produced: 26
Consumer 1 consumed: 26
Produced: 27
Consumer 2 consumed: 27
Produced: 28
Consumer 1 consumed: 28
Produced: 29
Consumer 2 consumed: 29
Produced: 30
Consumer 2 consumed: 30
Produced: 31
Produced: 32
Consumer 1 consumed: 31
Produced: 33
Produced: 34
Consumer 2 consumed: 32
Consumer 1 consumed: 33
Consumer 2 consumed: 34
Produced: 35
Consumer 1 consumed: 35

I52

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0, out = 0;

pthread_mutex_t mutex;
sem_t empty_slots;
sem_t full_slots;

int running = 1;

void* producer(void* arg) {
    int id = *(int*)arg;
    int value = 1;
    while (running) {
        sem_wait(&empty_slots);
        pthread_mutex_lock(&mutex);

        buffer[in] = value;
        printf("Producer %d produced: %d\n", id, value);
        in = (in + 1) % BUFFER_SIZE;
        value++;

        pthread_mutex_unlock(&mutex);
        sem_post(&full_slots);
        usleep(rand() % 500000);
    }
    return NULL;
}
```

```
void* consumer(void* arg) {
    while (running) {
        sem_wait(&full_slots);
        pthread_mutex_lock(&mutex);

        int value = buffer[out];
        printf("Consumer consumed: %d\n", value);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty_slots);

        usleep(rand() % 800000);
    }
    return NULL;
}
```

```
int main() {
    srand(time(NULL));

    pthread_t prod[2], cons;
    int prod_id[2] = {1, 2};

    pthread_mutex_init(&mutex, NULL);
    sem_init(&empty_slots, 0, BUFFER_SIZE);
    sem_init(&full_slots, 0, 0);

    for (int i = 0; i < 2; i++)
        pthread_create(&prod[i], NULL, producer, &prod_id[i]);

    pthread_create(&cons, NULL, consumer, NULL);

    sleep(10);
    running = 0;

    for (int i = 0; i < 2; i++)
        pthread_join(prod[i], NULL);
    pthread_join(cons, NULL);

    pthread_mutex_destroy(&mutex);
    sem_destroy(&empty_slots);
    sem_destroy(&full_slots);

    printf("Program finished.\n");
    return 0;
}
```

Result:

```
hitori@hitori:~$ gcc -O2 -Wall -lm -lpthread main.c -o p2cp
hitori@hitori:~$ ./p2cp
Producer 2 produced: 1
Producer 1 produced: 1
Consumer consumed: 1
Consumer consumed: 1
Producer 1 produced: 2
Producer 2 produced: 2
Producer 2 produced: 3
Consumer consumed: 2
Producer 1 produced: 3
Producer 2 produced: 4
Producer 2 produced: 5
Consumer consumed: 2
Producer 2 produced: 6
Consumer consumed: 3
Producer 1 produced: 4
Consumer consumed: 3
Producer 2 produced: 7
Consumer consumed: 4
Producer 1 produced: 5
Consumer consumed: 5
Producer 2 produced: 8
Consumer consumed: 6
Producer 2 produced: 9
Consumer consumed: 4
Producer 1 produced: 6
Consumer consumed: 7
Producer 2 produced: 10
Consumer consumed: 5
Producer 1 produced: 7
Consumer consumed: 8
Producer 1 produced: 8
Consumer consumed: 9
Producer 2 produced: 11
Consumer consumed: 6
Producer 1 produced: 9
Consumer consumed: 10
Producer 2 produced: 12
Consumer consumed: 7
Producer 1 produced: 10
Consumer consumed: 8
Producer 2 produced: 13
```

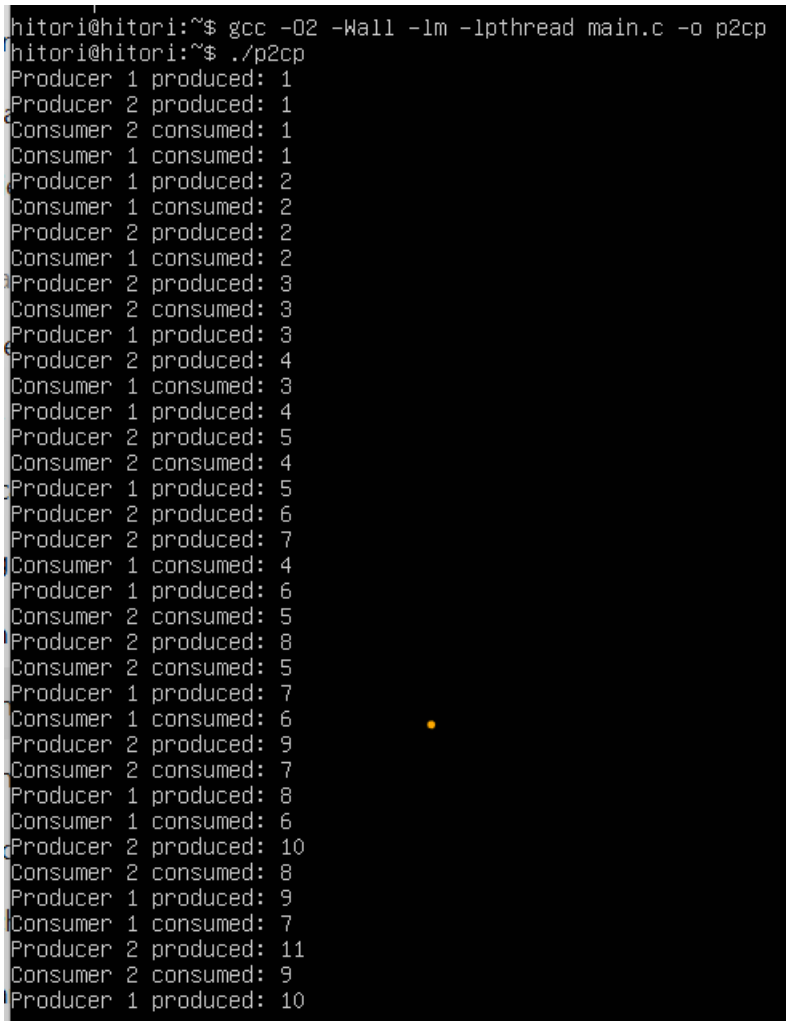
```
Consumer consumed: 11
Producer 1 produced: 11
Consumer consumed: 9
Producer 2 produced: 14
Consumer consumed: 12
Producer 1 produced: 12
Consumer consumed: 10
Producer 1 produced: 13
Consumer consumed: 13
Producer 2 produced: 15
Consumer consumed: 11
Producer 1 produced: 14
Program finished.
```

I53.

Code: Adjust in main for 2 consumer.

```
int main() {
    srand(time(NULL));

    pthread_t prod[2], cons[2];
    int prod_id[2] = {1, 2};
    int cons_id[2] = {1, 2};

    pthread_mutex_init(&mutex, NULL);
    sem_init(&empty_slots, 0, BUFFER_SIZE);
    sem_init(&full_slots, 0, 0);

    for (int i = 0; i < 2; i++)
        pthread_create(&prod[i], NULL, producer, &prod_id[i]);
    for (int i = 0; i < 2; i++)
        pthread_create(&cons[i], NULL, consumer, &cons_id[i]);

    sleep(10);
    running = 0;

    for (int i = 0; i < 2; i++)
        pthread_join(prod[i], NULL);
    for (int i = 0; i < 2; i++)
        pthread_join(cons[i], NULL);

    pthread_mutex_destroy(&mutex);
    sem_destroy(&empty_slots);
    sem_destroy(&full_slots);

    printf("Program finished.\n");
    return 0;
}
```

Result:


hitori@hitori:~$ gcc -O2 -Wall -lm -lpthread main.c -o p2cp
hitori@hitori:~$ ./p2cp
Producer 1 produced: 1
Producer 2 produced: 1
Consumer 2 consumed: 1
Consumer 1 consumed: 1
Producer 1 produced: 2
Consumer 1 consumed: 2
Producer 2 produced: 2
Consumer 1 consumed: 2
Producer 2 produced: 3
Consumer 2 consumed: 3
Producer 1 produced: 3
Producer 2 produced: 4
Consumer 1 consumed: 3
Producer 1 produced: 4
Producer 2 produced: 5
Consumer 2 consumed: 4
Producer 1 produced: 5
Producer 2 produced: 6
Producer 2 produced: 7
Consumer 1 consumed: 4
Producer 1 produced: 6
Consumer 2 consumed: 5
Producer 2 produced: 8
Consumer 2 consumed: 5
Producer 1 produced: 7
Consumer 1 consumed: 6
Producer 2 produced: 9
Consumer 2 consumed: 7
Producer 1 produced: 8
Consumer 1 consumed: 6
Producer 2 produced: 10
Consumer 2 consumed: 8
Producer 1 produced: 9
Consumer 1 consumed: 7
Producer 2 produced: 11
Consumer 2 consumed: 9
Producer 1 produced: 10